SLSQP-NMPC: Exact Derivatives and Exact Trajectory Reuse for Condensed Nonlinear Model Predictive Control Solver and Solution Engine

Kamran Turkoglu
Technical Note

Abstract

This note presents the selected Python SLSQP implementation for condensed nonlinear model predictive control (NMPC), with nonlinear dynamics, bounded inputs, and optional path inequalities. The compiled SLSQP core of Kraft (1988) [1] is retained, including its major iteration, BFGS approximation, merit line search, stopping rules, and globalization logic. Computational effort is reduced through an exact discrete adjoint, exact condensed path-constraint sensitivities, and byte-exact trajectory reuse at identical control vectors. Five NMPC problems give end-to-end speedups from 6.40 to 9.44, while independently evaluated feasibility and stationarity limits (computed after termination) are preserved. The geometric-mean speedup is 7.56 under the same Python runtime, solver implementation, numerical tolerances, and outer timing boundary.

1 Scope and selection

Nonlinear model predictive control requires repeated solutions of closely structured finite-horizon problems (often under a strict sampling interval and with limited computational capacity available at each sample). For the considered condensed formulation, most baseline work is spent in finite-difference trajectory simulations across every component of the horizon control vector at every major iteration. The selected implementation removes this repeated work without changing the SLSQP algorithm, its convergence tolerance, its accepted-step logic, its stopping modes, or its reported status.

The comparison uses one Python process and the same compiled SciPy SLSQP core (SciPy 1.13.1) described by SciPy Developers (2024) [2], rather than comparing Python code against compiled C++. Within each paired experiment, the model, initial point, bounds, constraints, convergence tolerance, iteration limit, and outer timing boundary remain fixed. The five-problem comparison (one complete solve) is timed from evaluator construction through the returned result. Each closed-loop comparison is timed by one outer clock interval from the sampled-state controller entry through problem assembly, the complete solve, success checking, control extraction, and preparation of the next warm start. The five-problem comparison uses `ftol`$= 10^{-11}$ and 600 major iterations. The mass-spring and aircraft studies use `ftol`$= 10^{-10}$ and 10^{-8}, respectively, with 300 major iterations. A candidate is admissible when every solve succeeds, the constraint violation is at most 10^{-9}, the relative objective difference is at most 10^{-10}, and both the scaled and the absolute independently recomputed KKT residuals are at most 2×10^{-6} under the fixed benchmark units. This rule governs the five-problem selection benchmark, and it is applied to every timed repeat rather than to one representative solve. The two closed-loop case studies report independent feasibility measurements but do not apply this KKT acceptance rule. The aircraft values exceed the 10^{-9} five-problem limit; its public solves therefore use an explicitly declared 10^{-7} post-solve feasibility tolerance. The certificate scores all four Karush–Kuhn–Tucker conditions, and it carries no second-order content, so a passing residual indicates an approximate first-order point and makes no local-optimality claim. Primal feasibility covers the input box as well as the path rows, dual feasibility is enforced and then measured, stationarity is the residual of the fitted multiplier system, and complementarity is the largest multiplier–slack product, reported separately for path rows and for bounds. Multipliers are fitted over every inequality under nonnegativity, and each multiplier is penalized by its own slack (its own primal residual), so activity is never inferred from a single absolute primal tolerance. The fitted system is column-equilibrated and the residuals are evaluated from the unscaled multipliers in the original units, because an unequilibrated fit fails to recover an existing multiplier once the constraint Jacobian is badly scaled. A bounded-variable active-set solve is used, since the trust-region alternative terminates on an interior tolerance and

leaves a floor on every multiplier, which would make the reported complementarity a property of the solver rather than of the point. The scaled residual controls unit sensitivity, while the absolute residual prevents a large fitted multiplier from hiding an uncancelled stationarity component in this fixed-unit benchmark.

The selected method has a lower median end-to-end time than this control on every test problem (using five batches of five timed solves after two warm-up solves). The comparison is between two engines: the exact-derivative evaluator and the value-only finite-difference control. Historical structured, inequality, and major-iteration candidates are not executed under this end-to-end contract, so no statement is made about them. This bounded result does not assert optimality within every implemented candidate, universal runtime optimality, worst-case execution time, or global optimality for a nonconvex control problem.

2 General NMPC formulation

Properly defining the general finite-horizon problem is required before its derivative structure can be used (independently of a specific plant or benchmark configuration used here). The condensed NMPC problem is given by

$$\begin{aligned} \min_{U} \quad & J(U) = \Phi(x_N) + \sum_{k=0}^{N-1} \ell_k(x_k, u_k), \\ \text{subject to} \quad & x_{k+1} = f_k(x_k, u_k), \qquad x_0 = \bar{x}, \\ & \underline{u}_k \le u_k \le \overline{u}_k, \qquad c_k(x_{k+1}, u_k) \ge 0. \end{aligned} \tag{1}$$

Here, $U = (u_0, \ldots, u_{N-1})$ is the condensed decision vector, x_k is the state, f_k is the discrete dynamics map, and c_k contains the path inequalities. The functions are assumed to be continuously differentiable over the iterates reached by SLSQP (including all accepted line-search points and derivative evaluation points used internally).

For the quadratic tracking cost used in the verification problems, the objective is given by

$$J(U) = \frac{1}{2} \sum_{k=0}^{N-1} \left[(x_k - x^\star)^\mathsf{T} Q_k (x_k - x^\star) + u_k^\mathsf{T} R_k u_k \right] + \frac{1}{2} (x_N - x^\star)^\mathsf{T} Q_N (x_N - x^\star). \tag{2}$$

Here, $x^\star$ is the target state, while Q_k, R_k, and Q_N are the state, input, and terminal weights, respectively.

3 Exact derivative reduction

With the problem defined, attention is now given to the repeated derivative calculations. The dynamics Jacobians are given by

$$A_k = \frac{\partial f_k}{\partial x}(x_k, u_k), \qquad B_k = \frac{\partial f_k}{\partial u}(x_k, u_k). \tag{3}$$

Here, A_k maps state perturbations and B_k maps control perturbations at stage k. The differentiation and local linearization follow the multivariable analysis construction given by Aleksandrov et al. (1963) [3].

The exact objective gradient is obtained by one reverse recurrence. The recurrence is given by

$$\lambda_N = Q_N (x_N - x^\star), \qquad \lambda_k = Q_k (x_k - x^\star) + A_k^\mathsf{T} \lambda_{k+1}, \qquad \nabla_{u_k} J = R_k u_k + B_k^\mathsf{T} \lambda_{k+1}. \tag{4}$$

Here, λ_k is the discrete costate, and the reverse recurrence is evaluated for $k = N-1, \ldots, 0$. Equation (4) can be obtained directly from the chain rule applied to Equation (1), without forming a dense state-to-control Jacobian.

The condensed state sensitivity is propagated during the same trajectory evaluation. Its recurrence is given by

$$S_0 = 0, \qquad S_{k+1} = A_k S_k + \mathcal{B}_k, \qquad \frac{\partial c_k}{\partial U} = c_{x,k} S_{k+1} + \mathcal{C}_{u,k}. \tag{5}$$

Here, $S_k = \partial x_k / \partial U$, while $\mathcal{B}_k$ and $\mathcal{C}_{u,k}$ place B_k and $c_{u,k}$ in the control block associated with stage k.

If one trajectory rollout has cost proportional to N, forward finite differences require $N n_u + 1$ rollouts for one objective gradient. Their dynamics work is therefore proportional to $N^2 n_u$ (for fixed stage evaluation cost). The

adjoint requires one forward rollout and one reverse sweep, so its dynamics work is proportional to N for fixed state and input dimensions during optimization.

The objective, gradient, constraint values, and constraint Jacobian are often requested at the same control vector (within one SLSQP major iteration). For this purpose, a contiguous copy of U is retained and compared through an allocation-free byte view. Reuse occurs only when the next argument has the same bytes. Thus, trajectory reuse introduces no approximation and no tolerance-dependent branch.

3.1 Why the unchanged SLSQP core is faster

The distinction between the optimizer core and the complete solve must be made explicit. The compiled SciPy implementation executes the same SLSQP major iteration, working-set treatment, BFGS update, line search, and stopping logic in both engines. However, that common core requests the objective, constraints, objective gradient, and constraint Jacobian from user callbacks at several control vectors during one solve. Equal optimizer code therefore does not imply equal callback work (or equal nonlinear model-evaluation cost) over one complete finite-horizon optimization at a controller sample.

Let $d = Nn_u$ denote the dimension of the condensed control vector, and let $\chi_c = 1$ when the nonlinear constraint Jacobian is also obtained by forward differences (otherwise $\chi_c = 0$). Before additional line-search evaluations, the approximate numbers of full-horizon rollouts at one derivative point are given by

$$r_{\mathrm{FD}} \approx (1 + \chi_c)(d + 1), \qquad r_{\mathrm{exact}} \approx 1. \tag{6}$$

Here, the first factor accounts for separately differenced objective and constraint callbacks (when both derivative arrays are absent from the solver call). Equation (6) is a callback-work model, rather than an SLSQP iteration-count model, and its count can be reduced without changing the major iteration. The actual measured counts also contain accepted or rejected line-search points and the callback ordering selected by the compiled solver wrapper during numerical optimization.

The resulting separation between compiled solver work and nonlinear prediction work is shown in Figure 1 for both evaluated callback paths. In the nominal path, SLSQP receives only objective and constraint values; the wrapper perturbs every component of U and repeats the complete nonlinear prediction for each required derivative column. In SLSQP-NMPC, one rollout stores x_k, A_k, and B_k (including every prediction stage). Equation (4) then gives the objective gradient by one reverse sweep, while Equation (5) gives the condensed constraint Jacobian by one forward sweep. All four callbacks reuse the stored trajectory when U is byte-identical, without changing any requested value or derivative supplied to the compiled core.

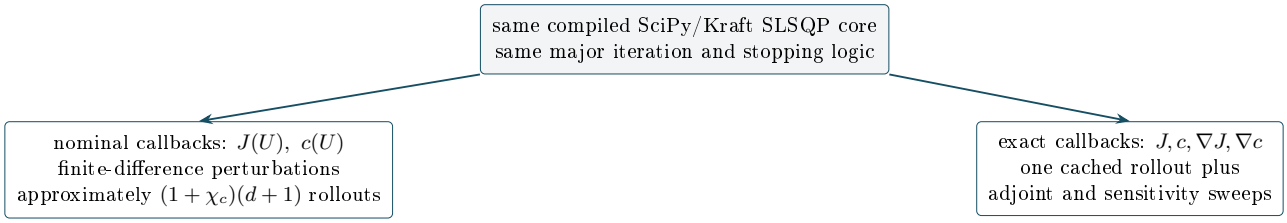


Figure 1: The SLSQP major-iteration core is common to both engines. The computational difference is the number of nonlinear prediction rollouts required by its callbacks.

A controlled mid-run comparison is given in Table 1. Cycle 19 is used to avoid the cold-start outlier at cycle 0. The condensed mass-spring problem has $d = 20$. Both engines terminate after six major iterations, while the number of complete full-horizon nonlinear prediction rollouts decreases from 253 to 7. Thus, the measured speedup cannot be attributed to fewer SLSQP iterations or a different compiled major-iteration algorithm on this controlled solve. It follows from a 36.14-fold reduction in nonlinear prediction work, together with lower callback overhead at the same requested solver tolerance. The nominal evaluator advances states only; it does not construct unused dynamics Jacobians.

The closed-loop evidence gives the same separation at controller scale. For the mass-spring case, both engines require 6.375 mean major iterations, while the corresponding mean rollout counts are 269.50 and 7.525 per controller cycle. For the aircraft case, the corresponding values are 21.000 and 20.889 iterations, but 747.75 and 60.222 rollouts. The

Table 1: Controlled mid-run evidence for the unchanged-core explanation.

Measure	Nominal SLSQP	SLSQP-NMPC	Nominal/selected
Condensed variables	20	20	1.00
Major iterations	6	6	1.00
Full-horizon rollouts	253	7	36.14
End-to-end controller-cycle time (ms)	441.6852	42.4192	10.41

major-iteration counts are therefore nearly unchanged, whereas the rollout reductions are 35.81 and 12.42. These counts explain the measured work but are not used as timing substitutes. The direct outer-clock median speedups are 7.72 and 5.00, respectively. Across the three order-balanced repetitions the paired per-repeat ratios span 7.45 to 8.56 and 4.18 to 4.96, so neither figure is reproducible to two decimals (three order-balanced repetitions).

This study does not establish that the compiled SLSQP core is faster. It establishes that the same core can solve these condensed NMPC problems faster when supplied with exact derivatives and exact trajectory reuse. The gain depends on the decision dimension, constraint structure, callback cost, and accepted line-search points under the specified convergence tolerance. Specifically, the aircraft controller-cycle median speedup is 5.00, and all five problems in Table 2 are below ten. The defensible release claim is therefore the direct measured result under each stated benchmark contract, rather than a universal tenfold bound.

4 Implementation

The complete release implementation is contained in `slsqp_nmpc.py` and has no benchmark-specific branch or model-name test. The public interface consists of the immutable `NMPCProblem` data class and the `solve_slsqp_nmpc` function. Model callbacks return the next state and the two matrices in Equation (3); an optional path callback returns c_k, $c_{x,k}$, and $c_{u,k}$ (with $c_k \geq 0$ denoting feasibility).

The SLSQP major iteration remains in the compiled SciPy implementation. Therefore, the BFGS update, merit line search, working-set treatment, stopping rules, and status modes can be retained without reimplementation. Only the mathematical information supplied to that implementation has been reduced and reused (at an identical control vector).

Each public solve creates one private evaluator and SciPy calls that evaluator synchronously. Concurrent public solves therefore use disjoint caches. Callback inputs are read-only views, while returned arrays are copied into preallocated contiguous storage and frozen. Weight matrices are tested to a roundoff-level symmetry tolerance and then symmetrized. Array dimensions are checked at every stage of every rollout, because an assignment alone is not a shape gate: a scalar or a row vector broadcasts silently into the destination. Finiteness is checked once on each assembled trajectory block. The initial rollout is validated strictly. A non-finite line-search trial is counted and returned to SLSQP, which may retreat, rather than being converted into a Python exception. Input bounds, initial-control feasibility, scalar or vector path values, callback return shapes, and weight definiteness (semidefinite state weights and a definite input weight) are also checked. The returned point is then re-established in full: the controls, the objective, its gradient, the trajectory, both dynamics Jacobians and, where path constraints are present, their values and both Jacobians. A single non-finite entry, or a diagnostic that cannot be completed, withdraws the reported success while every computed field is preserved, because a caller gating on that flag must not accept a point whose feasibility was never established.

For this purpose, we require every initial callback block to be finite (including the stored state and derivative arrays). Later non-finite trial values can be returned to SLSQP, which may retreat.

The computational sequence is as follows.

1. Roll out the nonlinear dynamics once and retain x_k, A_k, and B_k.
2. Evaluate the objective and propagate Equation (4) backward.
3. Propagate Equation (5) forward when path constraints are present.
4. Reuse the stored trajectory only for an identical control vector.

5. Pass the exact values and Jacobians to the unchanged SLSQP core.

5 Numerical verification

Implementation of the selected strategy on five NMPC problems is demonstrated next. For each problem, five batch medians are formed from five complete solves after two warm-up solves (which are not timed). The reported time is the median of those batch medians. Execution order alternates within each batch. Each row uses identical value models, starts, bounds, constraints, tolerances, and outer timing boundaries. The finite-difference path propagates states only, while the exact path additionally returns the Jacobians that it uses.

Table 2: End-to-end Python SLSQP comparison.

Problem	Variables	Finite diff. (ms)	Selected (ms)	Speedup	KKT residual
Double integrator	24	287.90	41.13	7.00	5.88×10^{-7}
Pendulum	24	305.25	40.55	7.53	4.70×10^{-7}
Van der Pol	22	229.08	35.78	6.40	9.43×10^{-7}
Duffing	22	268.79	34.76	7.73	3.99×10^{-7}
Unicycle with path constraint	28	612.79	64.93	9.44	2.86×10^{-7}

All repeated solves report success and zero measured constraint violation (using an independent post-solve evaluation). The largest relative objective difference is 3.18×10^{-14}, and the largest control infinity-norm difference is 5.36×10^{-6}. The selected implementation reduces the rollout count by factors from 23.04 to 56.25 across the five distinct control models.

The current release implementation has also been checked by a symbolic two-stage adjoint identity and 80 randomized finite-difference comparisons over all five model families. In addition, the release tests cover callback ownership and immutability, restricted-domain trials, diagnostic isolation, scalar and multirow constraints, multistate and multi-input derivatives, a stage-shift mutation, horizon-one behavior, warm starts, invalid data, feasibility, the controller-cycle timing boundary, timing aggregation, and bitwise agreement between serial and concurrent public solves. These checks establish derivative consistency and numerical agreement, but they do not establish global optimality for the nonlinear problems.

6 Closed-loop computational-cost accounting

The two case studies require a cost measure tied to the controller clock. One plant sample triggers one NMPC optimization (one controller update) in both implementations. Therefore, the per-sample time and the per-control-cycle time are identical in these experiments.

For an engine e evaluated over M closed-loop samples, the timing and work measures are given by

$$\bar{t}_e = \frac{1}{M}\sum_{j=1}^{M} t_{e,j}, \qquad T_e = \sum_{j=1}^{M} t_{e,j}, \qquad d_e = \frac{\bar{t}_e}{\Delta t}, \qquad \bar{i}_e = \frac{1}{M}\sum_{j=1}^{M} i_{e,j}, \qquad \bar{r}_e = \frac{1}{M}\sum_{j=1}^{M} r_{e,j}. \tag{7}$$

Here, $t_{e,j}$ is the complete controller-cycle compute time, T_e is its sum over the closed-loop run, Δt is the plant sampling interval, d_e is the mean compute-duty ratio, $i_{e,j}$ is the number of SLSQP major iterations, and $r_{e,j}$ is the number of full-horizon trajectory rollouts. The rollout count (including all prediction stages) is the principal model-evaluation cost.

The amortized time per major iteration is also reported as $T_e / \sum_j i_{e,j}$. This measured quantity includes callback work and compiled SLSQP work. It is not an asymptotic complexity estimate or a worst-case execution-time bound.

Each closed-loop case (mechanical and aircraft) is repeated three times. The engine order is cyclically permuted, so every engine occupies each execution position once. Each sample contains exactly one start and one stop of the outer controller-cycle timer. No component time (solver or callback) is subtracted or reconstructed. The time

reported at cycle j is the median of its three direct elapsed-time measurements. The first deterministic trajectory is retained for response reporting, after the state, control, iteration, and rollout histories have been verified equal across repetitions.

7 Nonlinear mass-spring-damper case study

The general formulation is now specialized to a nonlinear mechanical plant with one translational coordinate, one force input, and a constrained receding horizon. The plant equation is given by

$$m\ddot{q} + c\dot{q} + c_{\mathrm{n}}|\dot{q}|\dot{q} + kq + k_3 q^3 = u. \tag{8}$$

Here, q is the displacement, u is the applied force, $m = 1.2$ kg, $c = 0.35$ N s/m, $c_{\mathrm{n}} = 0.08$ N s^2/m^2, $k = 2.0$ N/m, and $k_3 = 0.8$ N/m^3. The cubic spring and velocity-dependent damping make the prediction model nonlinear over the complete simulated displacement and velocity range. The map $|\dot{q}|\dot{q}$ is continuously differentiable, but its second derivative is not defined at $\dot{q} = 0$; no second-order smoothness claim is used here.

The continuous model is discretized by fourth-order Runge–Kutta integration at 0.1 s (with a zero-order-held force over each sample). Exact discrete Jacobians are propagated through all four Runge–Kutta stages by applying the chain rule to every intermediate state. Their largest errors over 40 random central-difference checks are 9.79×10^{-10} for the state Jacobian and 5.92×10^{-10} for the input Jacobian.

The closed-loop experiment uses a 20-step prediction horizon and 40 control samples, which provide four seconds of controlled nonlinear motion. The initial state is $(q, \dot{q}) = (1, 0)$, the force is restricted to $|u| \leq 4$ N, and the nonlinear path constraint is $q^2 \leq 1.25^2$. Both engines use the same shifted warm start, prediction model, quadratic weights, convergence tolerances, nonlinear constraints, input limits, and compiled SLSQP core during all 40 samples.

The measured computational results are given in Table 3. Nominal SLSQP uses finite-difference objective and constraint derivatives, while SLSQP-NMPC uses the exact evaluations described in Section 3. Both times are measured by the same outer controller-cycle clock interval.

Table 3: Per-cycle and complete-run cost for the nonlinear mass-spring-damper case.

Measure	Nominal SLSQP	SLSQP-NMPC	Nominal/selected
Control samples/cycles	40	40	1.00
Sampling interval (ms)	100	100	1.00
Median time per cycle (ms)	405.39	52.51	7.72
Mean time per cycle (ms)	495.24	53.03	9.34
95th-percentile time per cycle (ms)	1124.17	100.60	11.17
Mean compute duty (%)	495.24	53.03	9.34
Mean major iterations per cycle	6.375	6.375	1.00
Mean trajectory rollouts per cycle	269.500	7.525	35.81
Amortized time per major iteration (ms)	77.68	8.32	9.34
Complete-run time (s)	19.809	2.121	9.34
Complete-run major iterations	255	255	1.00
Complete-run trajectory rollouts	10,780	301	35.81

Nominal SLSQP requires 4.95 sampling intervals on average for one controller update. SLSQP-NMPC requires 0.530 sampling intervals (53.03% mean duty). Its median and mean times remain below the 100 ms interval, while its 95th percentile is 100.60 ms. The complete 40-cycle rollout count decreases from 10780 to 301. The direct end-to-end median speedup is 7.72, and the three paired repetitions give 7.45 to 8.56.

The compute time at every controller cycle is shown in Figure 2. A logarithmic vertical scale is used to retain the sampling interval and both solver traces in one view.

The corresponding time-domain measures are given in Table 4. Both solutions satisfy the nonlinear path constraint at every sample.

The overlaid position, velocity, and force histories are shown in Figure 3. The maximum state and force differences are 7.88×10^{-8} and 1.43×10^{-6}, respectively.

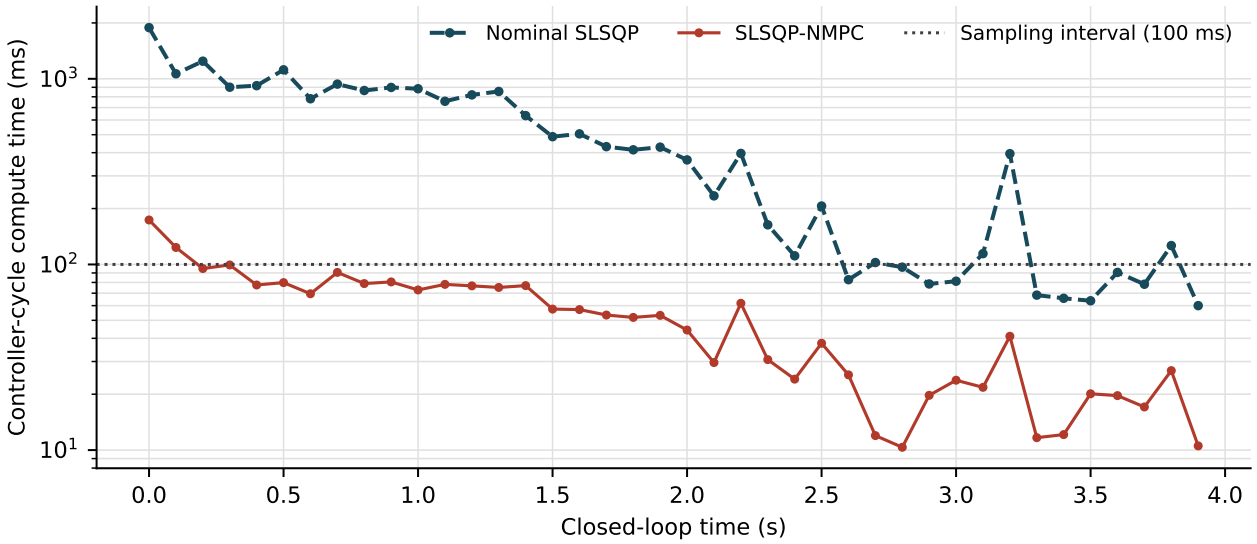


Figure 2: End-to-end controller-cycle compute time for the nonlinear mass-spring-damper case.

Table 4: Closed-loop response for the nonlinear mass-spring-damper case.

Measure	Nominal SLSQP	SLSQP-NMPC
Settling time (s)	1.60	1.60
Position integral absolute error (ms)	0.619356	0.619356
Position RMS (m)	0.326059	0.326059
Control RMS (N)	1.275417	1.275417
Final position (m)	2.7951×10^{-5}	2.7951×10^{-5}
Maximum constraint violation	0	0

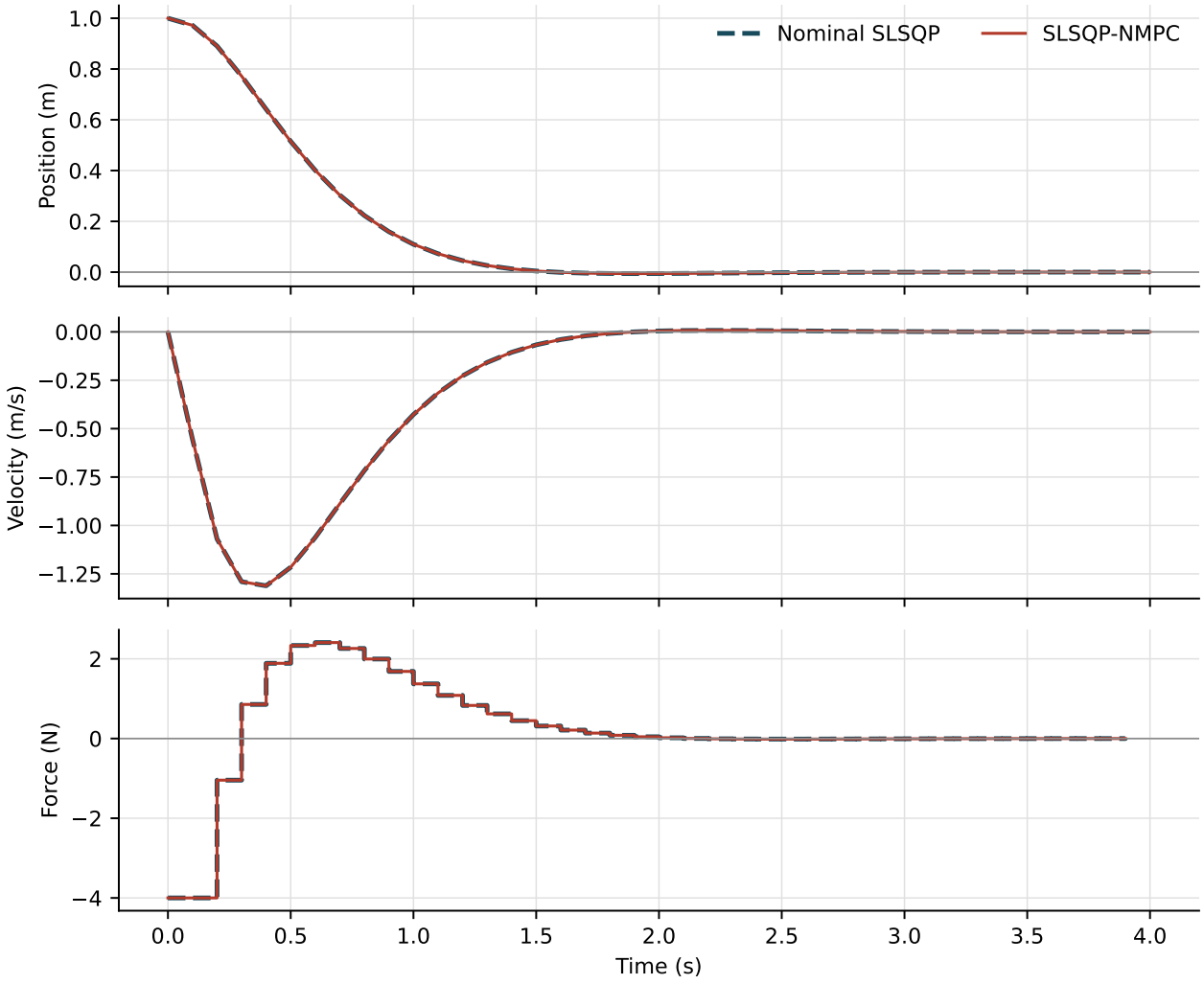


Figure 3: Time-domain comparison for the nonlinear mass-spring-damper NMPC case.

8 Nonlinear aircraft altitude and pitch case study

The second case considers an aircraft point-mass model restricted to the vertical X–Z plane. The kinematic and dynamic equations are given by

$$\begin{aligned} \dot{X} &= V\cos\gamma, & \dot{Z} &= V\sin\gamma, \\ \dot{V} &= \frac{T\cos\alpha - D}{m} - g\sin\gamma, & \dot{\gamma} &= \frac{L + T\sin\alpha}{mV} - \frac{g\cos\gamma}{V}. \end{aligned} \tag{9}$$

Here, X and Z are the horizontal position and altitude, V is the airspeed, γ is the flight-path angle, α is the angle of attack, and T is the thrust. The pitch response is reconstructed as $\theta = \gamma + \alpha$.

The nonlinear aerodynamic forces are given by

$$L = \frac{1}{2}\rho V^2 S C_L, \qquad D = \frac{1}{2}\rho V^2 S C_D, \qquad C_L = C_{L0} + C_{L\alpha}\alpha, \qquad C_D = C_{D0} + K C_L^2. \tag{10}$$

Here, L and D are the lift and drag, ρ is the air density, S is the wing area, and K is the induced-drag coefficient. The numerical model uses $m = 1200$ kg, $g = 9.81$ m/s^2, $\rho = 1.225$ kg/m^3, $S = 16$ m^2, $C_{L0} = 0.25$, $C_{L\alpha} = 4.8$ rad^{-1}, $C_{D0} = 0.025$, and $K = 0.055$.

The aircraft is initially trimmed at 60 m/s with $\gamma = 0$. The trim quantities satisfy $L + T\sin\alpha = mg$ and $T\cos\alpha = D$. The resulting angle of attack and thrust are 0.9923 deg and 1097.51 N, respectively. The NMPC input is an angle-of-attack increment, while the thrust remains at its trim value.

Fourth-order Runge–Kutta integration is applied at 0.25 s in both solver paths. The exact path also propagates the RK4 stage sensitivities. The 16-step prediction horizon is used over 36 closed-loop samples for a 30 m altitude command. The envelope restricts the speed to 42–82 m/s, the flight-path angle to ±16 deg, the angle of attack to −6–10 deg, and the altitude to −5–58 m. This aircraft result is reported at `ftol`$= 10^{-8}$ (the value used in both paths); no convergence claim is made for a tighter requested tolerance.

The largest central-difference errors of the discrete state and input Jacobians are 9.83×10^{-9} and 6.89×10^{-9}, respectively (computed before the closed-loop run). The computational comparison is given in Table 5.

Table 5: Per-cycle and complete-run cost for the nonlinear aircraft case.

Measure	Nominal SLSQP	SLSQP-NMPC	Nominal/selected
Control samples/cycles	36	36	1.00
Sampling interval (ms)	250	250	1.00
Median time per cycle (ms)	1708.58	341.88	5.00
Mean time per cycle (ms)	1775.46	367.72	4.83
95th-percentile time per cycle (ms)	2710.23	450.88	6.01
Mean compute duty (%)	710.18	147.09	4.83
Mean major iterations per cycle	21.000	20.889	1.01
Mean trajectory rollouts per cycle	747.750	60.222	12.42
Amortized time per major iteration (ms)	84.55	17.60	4.80
Complete-run time (s)	63.917	13.238	4.83
Complete-run major iterations	756	752	1.01
Complete-run trajectory rollouts	26,919	2,168	12.42

Nominal SLSQP requires 7.10 sampling intervals on average for one aircraft controller update. SLSQP-NMPC has a median time of 341.88 ms against the 250 ms sample interval. Its mean and 95th-percentile times are 367.72 and 450.88 ms, respectively. The median, the mean, and the 95th percentile all exceed the sampling interval, so this desktop result does not establish a deterministic real-time guarantee. The complete 36-cycle rollout count decreases from 26919 to 2168. The direct end-to-end median speedup is 5.00, and the three paired repetitions give 4.18 to 4.96.

The compute time at every aircraft controller cycle is shown in Figure 4. The sample-interval line (250 ms) shows the cycle deadline under the same end-to-end timing boundary.

The matched altitude and pitch measures are given in Table 6. The independently evaluated maximum violations are 5.12×10^{-8} for nominal SLSQP and 1.08×10^{-8} for SLSQP-NMPC. These values are reported directly; solver success is not used as a substitute for the external feasibility check.

The altitude, pitch, X–Z trajectory, and control histories are shown in Figure 5. The maximum state, control, and pitch differences are 3.97×10^{-6}, 3.03×10^{-7}, and 1.85×10^{-5} deg, respectively.

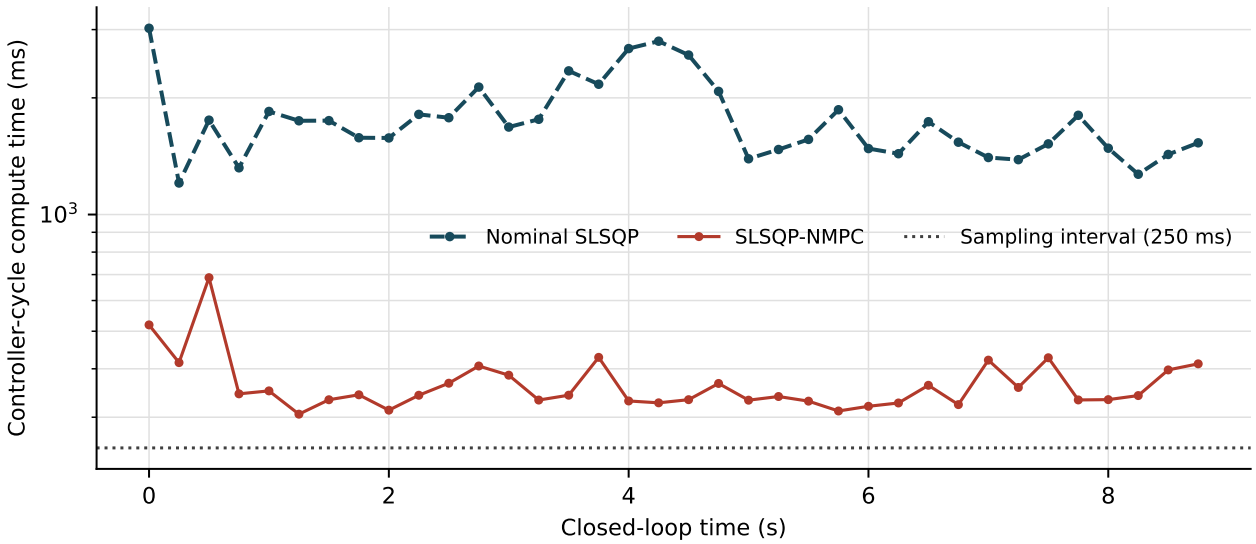


Figure 4: End-to-end controller-cycle compute time for the nonlinear aircraft altitude-control case.

Table 6: Closed-loop response for the nonlinear aircraft case.

Measure	Nominal SLSQP	SLSQP-NMPC
Settling time (s)	6.25	6.25
Altitude integral absolute error (m s)	49.308323	49.308309
Altitude RMS error (m)	10.441552	10.441552
Maximum altitude (m)	30.025805	30.025806
Final altitude (m)	29.096645	29.096647
Final speed (m/s)	54.755225	54.755225
Maximum pitch angle (deg)	20.495376	20.495376
Control RMS (deg)	3.144895	3.144895

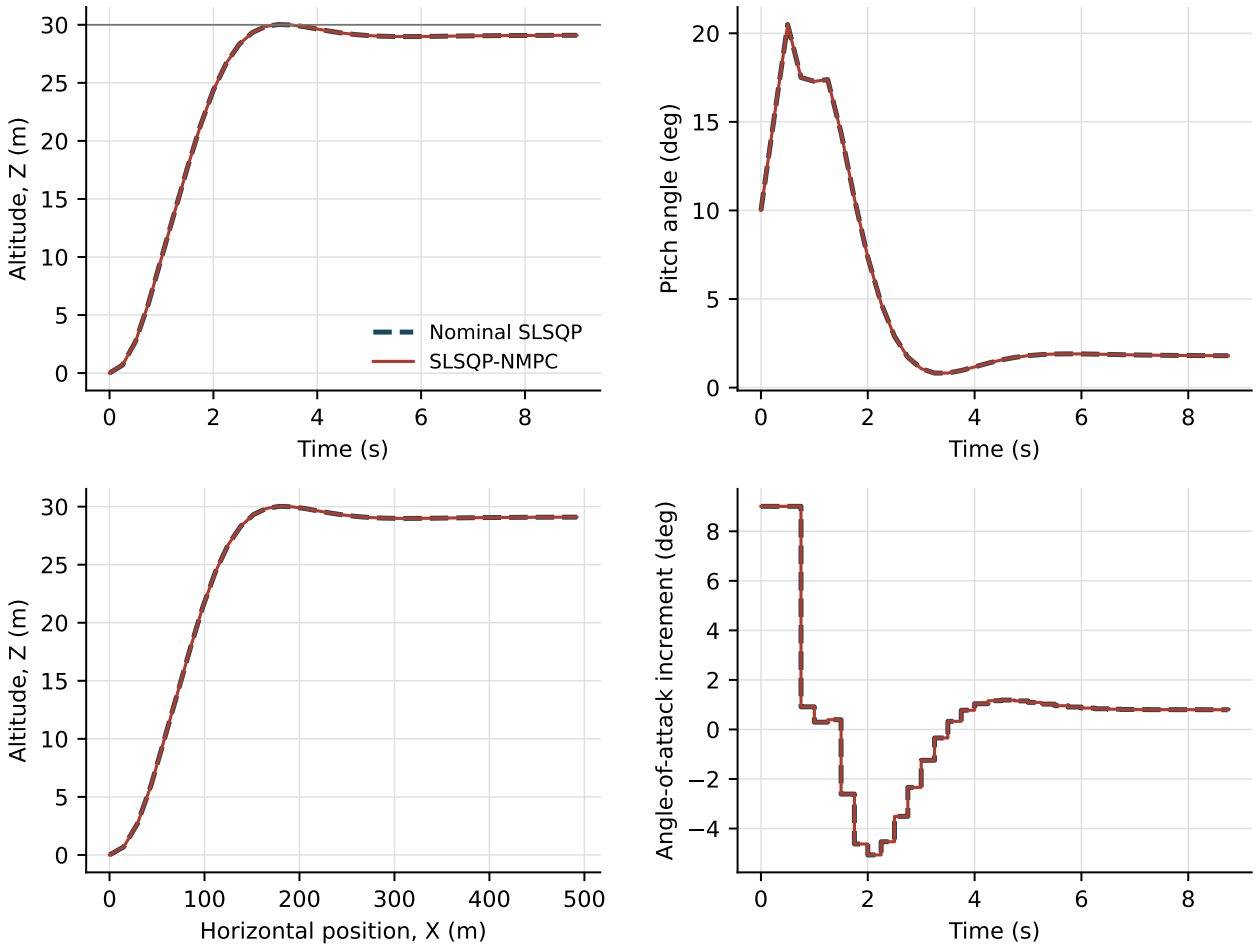


Figure 5: Time-domain and X–Z comparison for the nonlinear aircraft NMPC case.

9 Conclusion

In this study, we retained the established SLSQP algorithm and reduced the repeated mathematical work required to evaluate its condensed nonlinear program at every control sample. First, finite-difference objective gradients were replaced by the exact discrete adjoint for the complete horizon. Then, condensed path sensitivities and exact trajectory reuse were added without modifying the compiled solver core.

In this study, we also applied the same engine to nonlinear mechanical and aircraft altitude-control systems under matched numerical conditions.

The resulting implementation is faster than the stated value-only finite-difference control on every measured problem, under the recorded contract. The fresh, order-balanced mass-spring-damper and aircraft time-domain runs give direct controller-cycle median speedups of 7.72 and 5.00 against nominal finite differences, with matched response behavior. For larger multiple-shooting systems, sparse factorization may become preferable and should be evaluated at the measured crossover point.

References

[1] D. Kraft, *A Software Package for Sequential Quadratic Programming*, Technical Report DFVLR-FB 88-28, DLR German Aerospace Center, Institute for Flight Mechanics, Cologne, Germany, 1988.

[2] SciPy Developers, *SciPy 1.13.1 Reference Guide: SLSQP Optimization Interface*, SciPy software documentation and installed source, 2024.

[3] A. D. Aleksandrov, A. N. Kolmogorov, and M. A. Lavrent'ev (editors), *Mathematics: Its Content, Methods, and Meaning*, Volume I, MIT Press, Cambridge, Massachusetts, 1963.